

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: Framework JS: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Modalidad: Presencial / Laboratorio

Duración: 90 minutos

APELLIDOS Y NOMBRES: Alarcón Mendoza Estiven Rodrigo

Paso 1: Revisión del ciclo de renderizado con useState, React.StrictMode y Profiler

En este paso se configuró el entorno de React para observar el comportamiento del ciclo de renderizado de la aplicación. Se utilizó React.StrictMode para detectar posibles problemas durante el desarrollo y Profiler para medir el comportamiento de los renders sin depender únicamente de console.log.

El estado local se evidencia principalmente en el componente App.jsx, donde se controla la búsqueda y el filtro seleccionado mediante useState.

Ubicación del código

src/main.jsx
src/App.jsx

Código que evidencia el uso de React.StrictMode y Profiler

```
import React, { Profiler } from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import { ThemeProvider } from './context/ThemeContext.jsx'
import './index.css'

function onRenderCallback(
  id,
  phase,
  actualDuration,
  baseDuration,
  startTime,
  commitTime
) {
  console.table({
    componente: id,
    fase: phase,
    duracionActual: `${actualDuration.toFixed(2)} ms`,
    duracionBase: `${baseDuration.toFixed(2)} ms`,
    inicio: startTime.toFixed(2),
    commit: commitTime.toFixed(2),
  })
}
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <Profiler id="AstralArchiveApp" onRender={onRenderCallback}>
      <ThemeProvider>
        <App />
      </ThemeProvider>
    </Profiler>
  </React.StrictMode>,
)
```

Paso 2: Uso de useEffect, async/await y limpieza con AbortController

En este paso se implementó un Hook personalizado llamado useFetchCharacters, el cual simula la carga de personajes. Dentro de este Hook se utilizó useEffect para ejecutar un efecto secundario al montar el componente.

También se aplicó async/await y una función de limpieza mediante AbortController, lo cual permite cancelar una operación si el componente se desmonta antes de terminar la carga.

Ubicación del código

src/hooks/useFetchCharacters.js

Código que evidencia useEffect, async/await y AbortController

```
import { useEffect, useState } from 'react'
import { characters } from '../data/characters'

export function useFetchCharacters() {
  const [data, setData] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)

  useEffect(() => {
    const controller = new AbortController()

    async function fetchCharacters() {
      try {
        setLoading(true)

        await new Promise((resolve, reject) => {
          const timeout = setTimeout(resolve, 700)

          controller.signal.addEventListener('abort', () => {
            clearTimeout(timeout)
            reject(new DOMException('Solicitud cancelada', 'AbortError'))
          })
        })

        if (!controller.signal.aborted) {
```

```

        setData(characters)
      }
    } catch (err) {
      if (err.name !== 'AbortError') {
        setError('No se pudieron cargar los personajes')
      }
    } finally {
      if (!controller.signal.aborted) {
        setLoading(false)
      }
    }
  }
}

fetchCharacters()

return () => {
  controller.abort()
}
}, [])

return { data, loading, error }
}

```

Paso 3: Implementación de useContext para tema global

En este paso se implementó un contexto global llamado ThemeContext, el cual permite manejar el tema claro y oscuro de la aplicación sin necesidad de pasar propiedades manualmente entre componentes.

Esto evita el problema conocido como prop drilling, ya que cualquier componente dentro del ThemeProvider puede acceder al tema y a la función para cambiarlo.

Ubicación del código

```

src/context/ThemeContext.jsx
src/components/Navbar.jsx
src/main.jsx

```

Código que evidencia createContext, useContext y Provider

```

import { createContext, useContext, useEffect, useState } from 'react'

const ThemeContext = createContext(null)

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light')

  useEffect(() => {
    document.documentElement.setAttribute('data-theme', theme)
  }, [theme])
}

```

```

const toggleTheme = () => {
  setTheme((currentTheme) => (currentTheme === 'light' ? 'dark' : 'light'))
}

return (
  <ThemeContext.Provider value={{ theme, toggleTheme }}>
    {children}
  </ThemeContext.Provider>
)
}

export function useTheme() {
  const context = useContext(ThemeContext)

  if (!context) {
    throw new Error('useTheme debe usarse dentro de ThemeProvider')
  }

  return context
}

```

Código que evidencia el consumo del contexto

```

import { useState } from 'react'
import { useTheme } from '../context/ThemeContext'

export function Navbar({ favoritesCount }) {
  const { theme, toggleTheme } = useTheme()
  const [menuOpen, setMenuOpen] = useState(false)

  const closeMenu = () => setMenuOpen(false)

  return (
    <header className="navbar">
      <div className="navbar-top">
        <div className="brand">
          <div className="brand-icon" aria-hidden="true">
            ✦
          </div>
          <div>
            <h1>Astral Archive</h1>
            <span>React Hooks SPA</span>
          </div>
        </div>
      </div>

      <button
        type="button"

```

```

        className="nav-toggle"
        aria-expanded={menuOpen}
        aria-label={menuOpen ? 'Cerrar menú' : 'Abrir menú'}
        onClick={() => setMenuOpen((open) => !open)}
      >
        {menuOpen ? 'X' : '≡'}
      </button>
    </div>

    <div className={`navbar-menu ${menuOpen ? 'is-open' : ''}`}>
      <nav className="nav-links" onClick={closeMenu}>
        <a href="#home">Inicio</a>
        <a href="#search">Buscar</a>
        <a href="#favorites">Favoritos</a>
      </nav>

      <div className="nav-actions">
        <button type="button" className="ghost-btn" onClick={toggleTheme}>
          {theme === 'light' ? '☀️ Oscuro' : '🌙 Claro'}
        </button>

        <button type="button" className="login-btn">
          ♥ Favoritos ({favoritesCount})
        </button>
      </div>
    </div>
  </header>
)
}

```

Paso 4: Optimización del renderizado con useMemo, useCallback y useRef

En este paso se optimizó el comportamiento de renderizado de la aplicación. Se utilizó useMemo para evitar recalcular innecesariamente la lista de personajes filtrados, useCallback para evitar recrear funciones en cada render y useRef para manejar el enfoque del input de búsqueda sin generar nuevos renders.

Ubicación del Código

```

src/App.jsx
src/components/SearchPanel.jsx

```

Código que evidencia useMemo

```

import { useCallback, useEffect, useMemo, useReducer, useRef, useState } from 'react'
import { CharacterGrid } from '../components/CharacterGrid'
import { HeroBanner } from '../components/HeroBanner'
import { InfoPanel } from '../components/InfoPanel'
import { Navbar } from '../components/Navbar'

```

```

import { RecentSearches } from './components/RecentSearches'
import { SearchPanel } from './components/SearchPanel'
import { useFetchCharacters } from './hooks/useFetchCharacters'
import { useLocalStorage } from './hooks/useLocalStorage'
import {
  favoritesReducer,
  initialFavoritesState,
} from './reducer/favoritesReducer'

function App() {
  const { data: characters, loading, error } = useFetchCharacters()

  const [search, setSearch] = useState('')
  const [selectedElement, setSelectedElement] = useState('All')
  const [recentSearches, setRecentSearches] = useLocalStorage(
    'recent-searches',
    []
  )

  const [favoritesState, dispatch] = useReducer(
    favoritesReducer,
    initialFavoritesState
  )

  const searchInputRef = useRef(null)

  useEffect(() => {
    searchInputRef.current?.focus()
  }, [])

  useEffect(() => {
    if (search.trim().length < 3) return

    const timer = setTimeout(() => {
      setRecentSearches((currentSearches) => {
        const normalizedSearch = search.trim()
        const withoutRepeated = currentSearches.filter(
          (item) => item.toLowerCase() !== normalizedSearch.toLowerCase()
        )

        return [normalizedSearch, ...withoutRepeated].slice(0, 5)
      })
    }, 700)

    return () => clearTimeout(timer)
  }, [search, setRecentSearches])

  const filteredCharacters = useMemo(() => {
    const normalizedSearch = search.toLowerCase().trim()

```

```

return characters.filter((character) => {
  const matchesSearch =
    character.name.toLowerCase().includes(normalizedSearch) ||
    character.role.toLowerCase().includes(normalizedSearch) ||
    character.region.toLowerCase().includes(normalizedSearch)

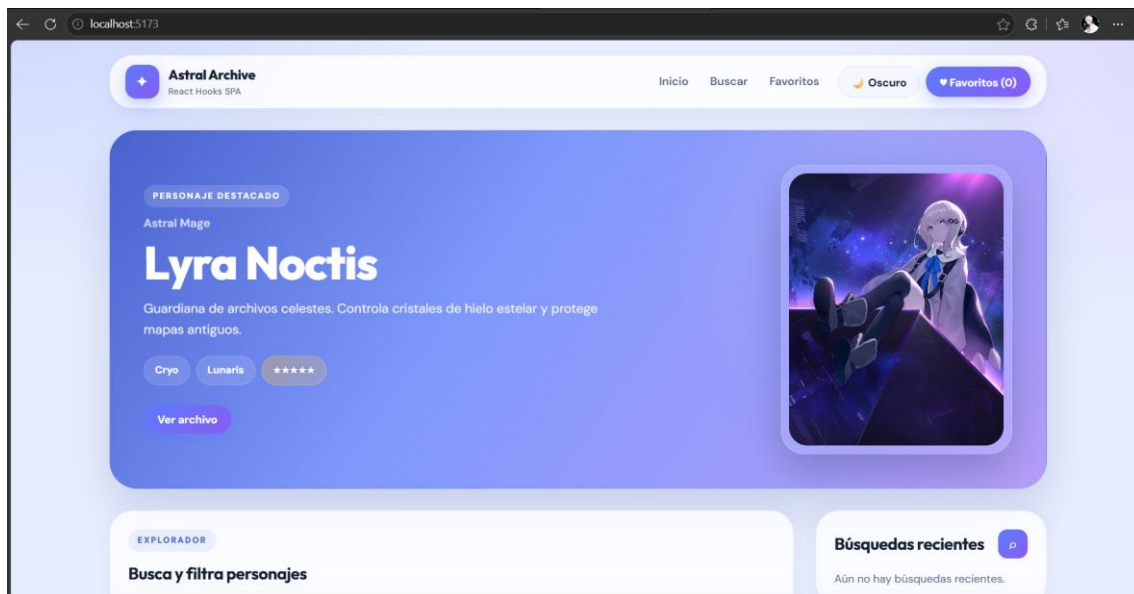
  const matchesElement =
    selectedElement === 'All' || character.element === selectedElement

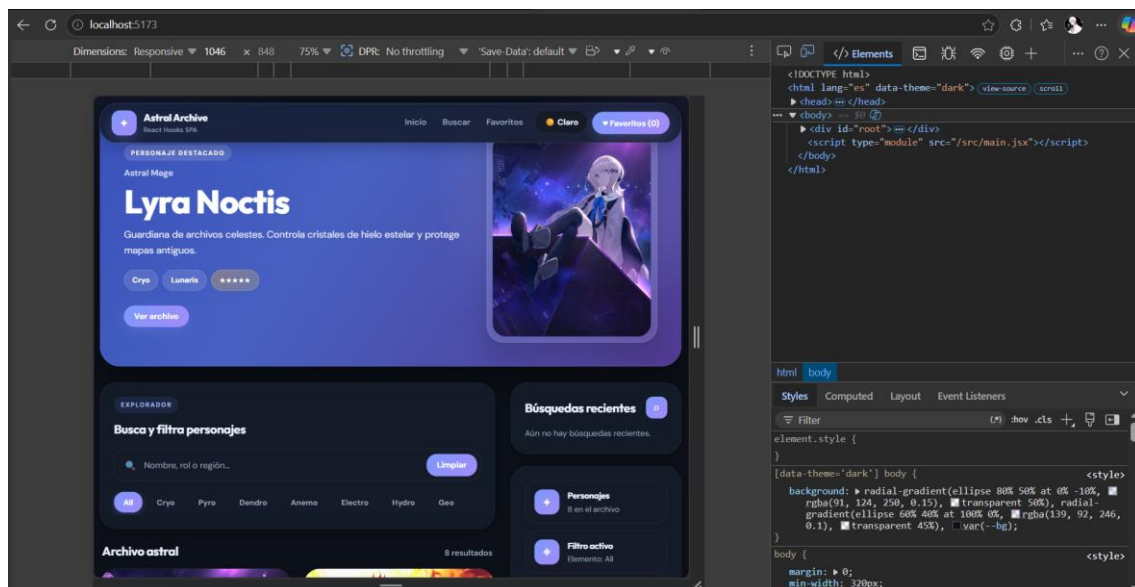
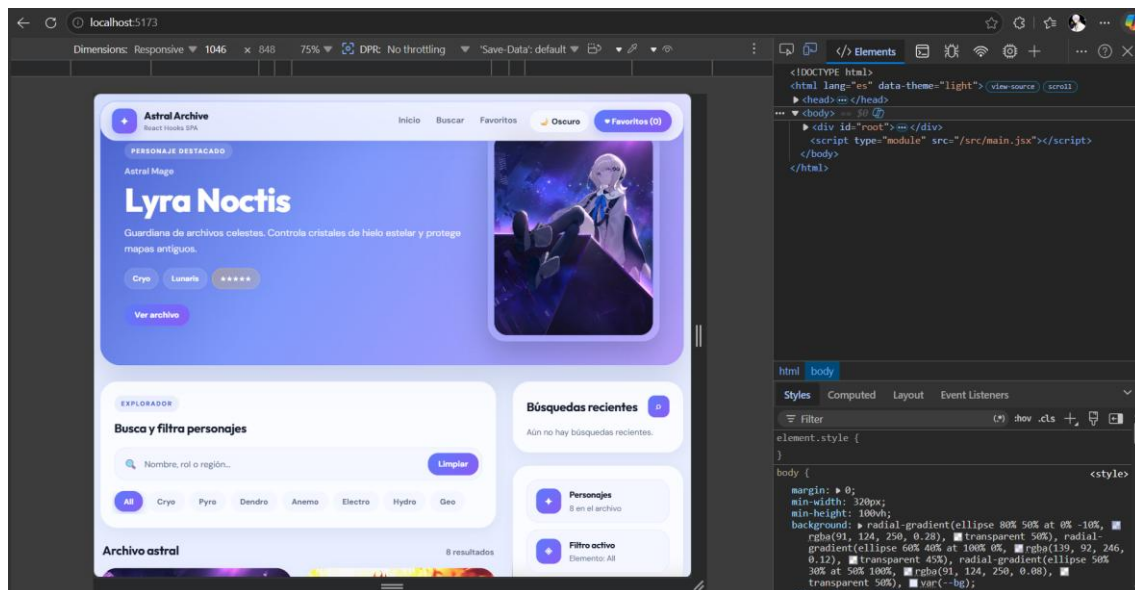
  return matchesSearch && matchesElement
})
}, [characters, search, selectedElement])

const featuredCharacter = useMemo(() => {
  return characters.find((character) => character.rarity === 5) || characters[0]
}, [characters])

```

Resultado:





Link de github: [Ichimaru465/Practica-S7-AME](https://github.com/Ichimaru465/Practica-S7-AME)